

Tehnička dokumentacija

AWS Cloud Deployment

Denis Sofović

May 30, 2026

Arhitekturni dijagram sistema

Arhitekturni dijagram sistema nalazi se u `docs` folderu pod nazivom `ims-diagram.png`.

Dijagram prikazuje kompletnu AWS infrastrukturu koja uključuje VPC sa dva AZ-a, javne i privatne pod mreže, Application Load Balancere, EC2 instance, RDS PostgreSQL bazu podataka, S3 bucket te Auto Scaling grupe.

Procjena troškova AWS servisa

Detaljna procjena troškova nalazi se u `docs` folderu pod nazivom `ims-cost-estimate.pdf`.

Softverski preduslovi

Lokalni alati

- **Terraform** — verzija 1.5+
- **AWS CLI** — verzija 2.x
- **Docker** — verzija 24+
- **Git**

AWS preduslovi

- Aktivan AWS Academy sandbox nalog
- AWS kredencijali (Access Key ID, Secret Access Key, Session Token)

Koraci za pokretanje ručnog deploymenta

1. Pokretanje sandbox okruženja

Otvoriti AWS Academy i pokrenuti lab sesiju klikom na *Start Lab*. Sačekati nekoliko minuta dok indikator ne postane zelen, zatim kliknuti na dugme *AWS* kako bi se pristupilo konzoli.

2. Kreiranje VPC infrastrukture

1. Otvoriti **VPC** servis i kreirati novi VPC:
 - Naziv: `ims-vpc`
 - CIDR: `172.31.0.0/16`

2. Kreirati 4 pod mreže (2 javne, 2 privatne) u `us-east-1a` i `us-east-1b`:
 - `ims-public-1` — `172.31.0.0/18` (`us-east-1a`)
 - `ims-public-2` — `172.31.64.0/18` (`us-east-1b`)
 - `ims-private-1` — `172.31.128.0/18` (`us-east-1a`)
 - `ims-private-2` — `172.31.192.0/18` (`us-east-1b`)
3. Kreirati **Internet Gateway** i attachati ga na VPC.
4. Kreirati **Elastic IP** adresu.
5. Kreirati **NAT Gateway** u pod mreži `ims-public-1`, odabrati opciju *Public* i dodijeliti Elastic IP.
6. Kreirati dvije route tabele:
 - `public-rt` — ruta `0.0.0.0/0` prema Internet Gatewayu; asocirati javne pod mreže
 - `private-rt` — ruta `0.0.0.0/0` prema NAT Gatewayu; asocirati privatne pod mreže

3. Kreiranje security grupa

Kreirati četiri security grupe unutar `ims-vpc`:

- `alb-sg` — Inbound: HTTP 80 od `0.0.0.0/0`, HTTPS 443 od `0.0.0.0/0`
- `frontend-sg` — Inbound: Custom TCP 3000 od `alb-sg`, SSH 22 od My IP
- `backend-sg` — Inbound: Custom TCP 8080 od `alb-sg`, SSH 22 od My IP
- `rds-sg` — Inbound: PostgreSQL 5432 od `backend-sg`

4. Kreiranje RDS baze podataka

1. Otvoriti **RDS** → *Create database*
2. Odabrati **PostgreSQL**, template **Easy Create, Free Tier**
3. Postaviti korisničko ime `postgres` i lozinku
4. U sekciji *Connectivity* odabrati `ims-vpc` i security grupu `rds-sg`
5. Sačekati da status postane *Available*
6. Zabilježiti endpoint baze (koristi se u connection stringu)

5. Kreiranje S3 bucketa

1. Otvoriti **S3** → *Create bucket*
2. Naziv: `ims-assets`
3. Odčekerati opciju *Block all public access*
4. Nakon kreiranja dodati bucket policy koji dozvoljava javni pristup
5. Uploadati sve fajlove iz `assets` foldera u bucket

6. Kreiranje Application Load Balancera

Kreirati dvije target grupe:

- **frontend-tg** — protocol HTTP, port 3000, health check path `/health`
- **backend-tg** — protocol HTTP, port 8080, health check path `/`

Kreirati dva Application Load Balancera (*EC2 → Load Balancers → Create Load Balancer*):

- **frontend-alb** — Internet-facing, oba javna pod mreža, **alb-sg**, listener HTTP 80 → **frontend-tg**
- **backend-alb** — Internet-facing, oba javna pod mreža, **alb-sg**, listener HTTP 80 → **backend-tg**

7. Kreiranje Launch Templates

Backend Launch Template (*EC2 → Launch Templates → Create launch template*):

- Naziv: **backend-template**
- AMI: Ubuntu 22.04
- Instance type: **t2.small**
- Key pair: **vockey**
- Security group: **backend-sg**

User data skripta: Dostupna u **scripts** folderu

Frontend Launch Template:

- Naziv: **frontend-template**
- AMI: Ubuntu 22.04
- Instance type: **t2.small**
- Key pair: **vockey**
- Security group: **frontend-sg**

User data skripta: Dostupna u **scripts** folderu

8. Kreiranje Auto Scaling Group

Backend ASG:

1. Otvoriti *EC2 → Auto Scaling Groups → Create Auto Scaling Group*
2. Naziv: **backend-asg**, Launch template: **backend-template**
3. VPC: **ims-vpc**, Subnets: oba privatna subneta
4. Load balancing: prikačiti na **backend-tg**, Health check type: **ELB**
5. Group size: Desired **2**, Minimum **2**, Maximum **4**
6. U *Advanced configuration* postaviti:

- Health check grace period: **900** sekundi
- Default cooldown: **500** sekundi

7. Scaling policy: Target tracking, CPU utilization, target **60%**, estimated instance warmup: **600** sekundi

Frontend ASG:

1. Otvoriti *EC2* → *Auto Scaling Groups* → *Create Auto Scaling Group*
2. Naziv: **frontend-asg**, Launch template: **frontend-template**
3. VPC: **ims-vpc**, Subnets: oba javna subneta
4. Load balancing: prikvačiti na **frontend-tg**, Health check type: ELB
5. Group size: Desired **1**, Minimum **1**, Maximum **4**
6. U *Advanced configuration* postaviti:
 - Health check grace period: **900** sekundi
 - Default cooldown: **500** sekundi
7. Scaling policy: Target tracking, CPU utilization, target **80%**, estimated instance warmup: **600** sekundi

Scale down politika (za oba ASG-a):

1. *Automatic scaling* → *Create dynamic scaling policy*
2. Policy type: Simple scaling, naziv: **scale-down-policy**
3. Kreirati CloudWatch alarm: CPU < 30%, naziv: **low-cpu-alarm**
4. Action: Remove 1 capacity unit

9. CloudWatch monitoring

Dashboard:

1. *CloudWatch* → *Dashboards* → *Create dashboard*, naziv: **ims-dashboard**
2. Dodati widget: Line → EC2 → Per-Instance Metrics → CPUUtilization
3. Ponoviti za NetworkIn i NetworkOut

10. Provjera i pristup aplikaciji

Nakon što su sve instance u statusu *healthy* u target grupama, aplikaciji se pristupa putem:

`http://<frontend-alb-dns>`

Koraci za pokretanje Terraform deploymenta

Koraci za Terraform depolyment se nalaze u README.md.

Kratak opis izazova i rješenja

Problem 1: Ručno kreiranje RDS baze podataka

Opis problema: Ovaj problem se javio isključivo kod ručnog deploymenta aplikacije, gdje AWS nije dozvoljavao odabir storage tipa, VPC-a i instance tipa tokom kreiranja baze podataka.

Rješenje: Jedina opcija koja je bila dostupna bila je kreiranje baze putem *Easy Create* konfiguracije, međutim ta opcija nije omogućavala odabir prilagođenog VPC-a već je koristila isključivo defaultni. Kako bi se ovaj problem zaobišao, defaultni VPC je prilagođen vlastitim potrebama projekta, te je korišten umjesto posebno kreiranog VPC-a.

Problem 2: Kreiranje S3 bucketa pomoću Terraforma

Opis problema: Ovaj problem se javio tokom Terraform automatizacije deploymenta. Nakon pokretanja konfiguracije i uspješnog kreiranja svih ostalih resursa, Terraform bi izbacio sljedeću grešku:

```
Error: reading S3 Bucket object lock configuration:
operation error S3: GetObjectLockConfiguration,
https response error StatusCode: 403,
api error AccessDenied: User is not authorized to perform:
s3:GetBucketObjectLockConfiguration on resource:
"arn:aws:s3:::ims-assets-sofovic" with an explicit deny
in an identity-based policy
```

```
with aws_s3_bucket.assets,
on s3.tf line 1, in resource "aws_s3_bucket" "assets":
1: resource "aws_s3_bucket" "assets" {
```

Uzrok greške je ograničenje AWS Academy sandbox okruženja koje zabranjuje pristup `s3:GetBucketObjectLockConfiguration` operaciji, koju Terraform automatski poziva nakon kreiranja S3 bucketa.

Rješenje: Nije pronađeno tehničko rješenje unutar Terraform konfiguracije koje bi u potpunosti zaobislo ovo ograničenje sandboxa. Kao privremeno rješenje, S3 bucket se kreira ručno putem AWS konzole, a fajlovi se uploaduju manualno.